# Python for Engineers Activities - Lists

## Contents

Python for Engineers Activities - Lists ..... 1
DRY ..... 1
Online Tutorials ..... 1
Tutorial 1: Traversing a List with a Loop ..... 2
Tutorial 2: List and String Slicing ..... 3
Tutorial 3: List Functions ..... 6
in and not in ..... 6
Tutorial 4: List Methods ..... 7
Assignment 1: List Slicing and Iteration ..... 8
Assignment Submission ..... 8

Time required: 90 minutes

## DRY

Don't Repeat Yourself

## Online Tutorials

Go through the following tutorials.

- LearnPython.org Lists
- Python Lists
  - Access List Items
  - Change List Items
  - Add List Items
  - Remove List Items
  - Loop Lists
  - Sort Lists

- Copy Lists
- Join Lists
- List Methods

## Tutorial 1: Traversing a List with a Loop

A List is a sequence of elements. A String is a sequence of elements. A Tuple is a sequence of elements. The major difference is that a List is mutable, it can be changed. Otherwise, each of these sequences behave in similar ways.

The most common way to iterate or traverse the elements of a list is with a for loop. The syntax is the same as for strings.

```
cheeses = ["cheddar", "colby", "swiss"]
for cheese in cheeses:
    print(cheese)
cheddar
colby
swiss
```

If you want to write or update the elements, you need the indices. A common way to do that is to combine the functions **range** and **len**:

```
# range_len.py
numbers = [1, 3, 5]
for i in range(len(numbers)):
    # Update element in list
    numbers[i] = numbers[i] * 2
    print(f"{i + 1} {numbers[i]}")
```

This loop traverses the list and updates each element. **len** returns the number of elements in the list. **range** returns a list of indices from 0 to n − 1, where **n** is the length of the list. Each time through the loop, **i** gets the index of the next element. The assignment statement in the body uses **i** to read the old value of the element and to assign the new value.

Create a Python program named **list_loops.py**

```python
"""
    Name: list_loops.py
    Author:
    Created:
    Purpose:
"""
# Define a list of numbers: 1, 3, 5
numbers = [1, 3, 5]

# Iterate through each number in the list and print it
for number in numbers:
    print(f"{number}")

# Iterate through the indices of the list using a for loop
for i in range(len(numbers)):
    # Multiply each number in the list by 2
    numbers[i] = numbers[i] * 2

    # Print the modified numbers along with their respective indices
    print(f"{i + 1}. {numbers[i]}")
```

Example run:

```
1
3
5
1. 2
2. 6
3. 10
```

## Tutorial 2: List and String Slicing

The Python slice operator, represented by the colon **:**, allows you to extract a portion of a sequence like a string, list, or tuple by specifying a start, stop, and step size within square brackets [ ].

The general syntax for slicing is: **sequence[start:stop:step]**

```
new_list = original_list[start:stop:step]
new_string = original_string[start:stop:step]
```

- **start (optional)**: The index where the slice begins (inclusive). If omitted, it starts from the beginning of the sequence (index 0).

- **stop (optional)**: The index where the slice ends (exclusive). If omitted, it goes until the end of the sequence.
- **step (optional)**: The increment between indices while slicing. If omitted, it defaults to 1.

```
t = ["a", "b", "c", "d", "e", "f"]
print(t[1:3])
# Output: ['b', 'c']

print(t[:4])
# Output: ['a', 'b', 'c', 'd']

print(t[3:])
# Output: ['d', 'e', 'f']
```

- `t[1:3]` slices the index 1 and 2 items.
- `t[:4]` slices everything from the beginning of the list to index 3.
- `t[3:]` slices everything after and including index 3.

If you omit the first index, the slice starts at the beginning. If you omit the second, the slice goes to the end. If you omit both, the slice is a copy of the whole list.

```
print(t[:])
# Output: ['a', 'b', 'c', 'd', 'e', 'f']
```

Lists and strings can be sliced in similar ways. Create a Python program named **list_slice.py**

```python
"""
    Name: list_slice.py
    Author:
    Created:
    Purpose: Demonstrate Python slicing
"""

# Example 1: Slicing a range from a list
numbers = [1, 2, 3, 4, 5]
subset = numbers[1:4]
print(subset)

# Example 1: Slicing a range from a string
text = "PythonProgramming"
subset = text[0:6]
print(subset)
```

Example run:

```
[2, 3, 4]
Python
```

```python
# Example 2: Slicing with step
even_numbers = numbers[::2]
print(even_numbers)

# Example 2: Slicing with step
every_second_char = text[::2]
print(every_second_char)  # Output: PtoPormig

# Example 3: Slicing from the end
last_three = numbers[-3:]
print(last_three)

# Example 4: Modifying elements
fruits = ["apple", "banana", "cherry", "date"]
fruits[1:3] = ["pear", "grape"]
print(fruits)
```

Example run:

```
[1, 3, 5]
PtoPormig
[3, 4, 5]
['apple', 'pear', 'grape', 'date']
```

## Tutorial 3: List Functions

There are many built-in functions that operate on lists without writing your own loops.

| Function | Description |
|---|---|
| **len()** | Returns the number of items in the list |
| **sum()** | Returns the sum of the items in the list |
| **min()** | Returns the minimum of the items in the list |
| **max()** | Returns the maximum of the items in the list |
| **sorted()** | Returns a sorted list |

### in and not in

The in operator tells you if a list or string contains something. The not in operator tells you if the list of string doesn't contain something.

Create a Python file named **list_functions.py**

```python
nums = [3, 41, 12, 9, 74, 15]

print(len(nums))

print(max(nums))

print(min(nums))

print(sum(nums))

print(sum(nums) / len(nums))

print(sorted(nums))

# The in operator tells you if your list or string contains an element
if 3 in nums:
    print("Your list contains the number 3. ")

# not in tells you if your list or string does not contain an element
if 0 not in nums:
    print("Your list has no zeroes.")
```

Example run:

```
6
74
3
154
25.666666666666668
[3, 9, 12, 15, 41, 74]
Your list contains the number 3.
Your list has no zeroes.
```

## Tutorial 4: List Methods

Python provides methods that operate on lists. Methods use the **.** (dot) operator.

| Method | Description |
|---|---|
| **append(x)** | add **x** to the end of the list |
| **sort(x)** | sort the list in place |
| **count(x)** | return the number of times **x** occurs in the list |
| **index(x)** | return the location of the first occurrence of **x** |

| | |
|---|---|
| **reverse()** | reverse the list in place |
| **clear()** | remove all the elements from the list |
| **remove(x)** | remove the first occurrence of **x** from the list |
| **pop(p)** | remove the item at index **p** and return its value |
| **insert(p, x)** | insert **x** at index **p** of the list |

## Assignment 1: List Slicing and Iteration

Objective: Understand list slicing and iteration.

1. Create a list named numbers with the numbers from 1 to 10.
2. Print the first five numbers using list slicing.
3. Print the last three numbers using list slicing.
4. Use a loop to print each number in the list.
5. Use a loop to print each number in the list that is greater than 5.

Example run:

```
[1, 2, 3, 4, 5]
[8, 9, 10]
1
2
3
4
5
6
7
8
9
10
6
7
8
9
10
```

## Assignment Submission

1. Attach the program files.
2. Attach screenshots showing the successful operation of the programs.

3. Submit in Blackboard.